



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE





Warm up: API



Application Programming Interface (API) é

Um “contrato” documentado entre o chamador (deseja usar um serviço fornecido) e o chamado



O que uma API deve especificar?

Como o chamador identifica e localiza a função a ser chamada?

Como passar argumentos obrigatórios e opcionais?

Como o chamador recebe o valor de retorno?

O que acontece em caso de erro no chamador?



Identificar o chamador?

SaaS: endpoint URI recebe solicitação HTTP

rota consiste em URI base, método, caminho, argumentos opcionais e obrigatórios

operação geralmente primeiro componente de caminho do URI

por exemplo, docs em developers.themoviedb.org

GET /search/movie?params

GET /movie/:movie_id



Passagem de parâmetro?

Para GET, incorpore na string de consulta da URI

Para POST, pode ser enviado como parte do corpo do formulário.

```
GET /search/movie?query=Coco
```

URI base, versão da API

GET `https://api.themoviedb.org/3/search/movie?query=Coco`

Método

operação

parâmetros

```
GET https://api.themoviedb.org/3/search/movie?query=Coco&api_key=...
```



Obtendo resultados no retorno: JSON (JavaScript Object Notation)

- JSON object = hash não ordenado em { }
- Chaves são strings; valores podem ser string, number, [Array], ou {hash aninhado}
- Par chave/valor separado por vírgulas; espaço em branco é irrelevante

```
{ "title": "Coco",  
  "rating": "G",  
  "genres": [{ "id": 16, "name": "Animation" },  
             { "id": 10751, "name": "Family" }],  
  "vote_average": 8.1  
}
```



E agora?

API docs: `GET /movie/:movie_id`

retorna detalhes de um filme específico

De onde vem o valor de `movie_id`?

1. Você procura nos documentos da API
2. De uma chamada anterior para `/movie/search`
3. É apenas o título do filme (ou uma substring do título do filme)



E agora?

API docs: `GET /movie/:movie_id`

retorna detalhes de um filme específico

Para obter um URI totalmente qualificado para esta chamada, precisamos adicionar todos os seguintes itens:

- `api_key=...` parâmetro na URI
- Parâmetros adicionais da consulta (argumentos) na URI
- URI base



GET `https://api.themoviedb.org/3/movie/354912?api_key=...`

Princípios de design de software

A funcionalidade é o mínimo. O que separa um sistema que dura de um sistema que colapsa é a forma como ele é organizado por dentro. Bem-vindos à primeira aula de projeto de sistemas, a partir de hoje, vocês constroem.



O que vamos ver hoje

Esta aula tem uma provocação no centro: **código que funciona não é suficiente**. Vamos explorar os princípios que transformam um sistema funcional em um sistema sustentável — e conectar tudo isso ao projeto real que começa esta semana.

01

Bloco 1 — Provocação

O exercício dos dois códigos: ambos funcionam, mas apenas um é sustentável.

03

Bloco 3 — Projeto Real

Estrutura de pastas, o primeiro ADR do Compasso e as decisões que a equipe vai tomar no Workshop de Arquitetura.

02

Bloco 2 — Princípios

Separação de responsabilidades, coesão, acoplamento e nomes significativos.

04

Bloco 4 — Fechamento

Questão estruturante, reflexão coletiva e ponte para a próxima aula.



Objetivos de Aprendizagem

Ao final desta aula, você será capaz de:

1

Trabalho em equipe

Explicar por que a organização interna do código afeta diretamente a colaboração e a produtividade do time.

2

Coesão

Diferenciar coesão alta de coesão baixa com exemplos práticos e identificar problemas em código real.

3

Acoplamento

Reconhecer acoplamento excessivo e argumentar por que ele dificulta a manutenção ao longo do tempo.

4

Estrutura do Projeto

Propor uma separação de responsabilidades aplicável ao projeto real da equipe.





Bloco 1 — A Provocação

"Se o código funciona, por que importa como ele está organizado?"

Pense por um momento antes de responder. Esta pergunta vai guiar os próximos 20 minutos.



Dois Códigos, Um Problema

Os dois trechos abaixo fazem exatamente a mesma coisa: **cadastrar um usuário e enviar um e-mail de boas-vindas**. Leia os dois em silêncio por 3 minutos antes de continuar.

Versão A — Tudo junto

```
app.post('/register', async (req, res) => {
  const { name, email, password } = req.body;
  if (!name || !email || !password)
    return res.status(400).json({ error: 'Campos obrigatórios' });
  if (password.length < 8)
    return res.status(400).json({ error: 'Senha fraca' });
  const existing = await db.query(
    'SELECT * FROM users WHERE email = $1', [email]);
  if (existing.rows.length > 0)
    return res.status(409).json({ error: 'Email já cadastrado' });
  const hash = await bcrypt.hash(password, 10);
  const result = await db.query(
    'INSERT INTO users (name, email, password)
    VALUES ($1, $2, $3) RETURNING id', [name, email, hash]);
  const userId = result.rows[0].id;
  await transporter.sendMail({
    from: 'noreply@app.com', to: email,
    subject: 'Bem-vindo!',
    html: `<h1>Olá ${name}!</h1>
    <p>Sua conta foi criada.</p>`
  });
  res.status(201).json({ id: userId, name, email });
});
```

Versão B — Responsabilidades

```
// userValidator.js
function validateRegistration({ name, email, password }) {
  if (!name || !email || !password) return 'Campos obrigatórios';
  if (password.length < 8) return 'Senha fraca';
  return null;
}
```

```
// userService.js
async function registerUser({ name, email, password }) {
  const exists = await userRepository.findByEmail(email);
  if (exists) throw new ConflictError('Email já cadastrado');
  const hash = await bcrypt.hash(password, 10);
  return userRepository.create({ name, email, password: hash });
}
```

```
// emailService.js
async function sendWelcome(user) {
  await transporter.sendMail({
    from: 'noreply@app.com', to: user.email,
    subject: 'Bem-vindo!',
    html: templates.welcome(user)
  });
}
```

```
// userController.js
app.post('/register', async (req, res) => {
  const error = validateRegistration(req.body);
  if (error) return res.status(400).json({ error });
  const user = await registerUser(req.body);
  await sendWelcome(user);
  res.status(201).json(user);
});
```



A Pergunta que Revela Tudo

"Os dois fazem a mesma coisa. Agora imaginem: vocês precisam mudar o template do e-mail de boas-vindas. Em qual versão fariam isso com mais confiança? Por quê?"

Versão A — O problema

Para mudar o e-mail, você abre um arquivo com **30+ linhas** que misturam validação, banco de dados e e-mail. Qualquer toque pode quebrar algo completamente diferente.

Versão B — A solução

Você vai direto em `emailService.js`. O resto do sistema **nem precisa ser aberto**. Você sabe exatamente onde o problema está e qual é o impacto da mudança.





O Problema do Trabalho em Equipe

Agora eleve a escala. Dois colegas trabalham ao mesmo tempo: um adiciona validação de CPF, outro atualiza o template de e-mail.

Na Versão A

Os dois mexem no **mesmo arquivo**. Git detecta conflito. Alguém vai ter que resolver o merge — e pode introduzir um bug no processo. Isso é atrito real, semana após semana.

Na Versão B

Cada um mexe no **seu arquivo**: `userValidator.js` e `emailService.js`. Zero conflito de merge. A equipe avança em paralelo sem se bloquear mutuamente.

- ❑ Este não é um problema teórico. É o que vai acontecer com a equipe de vocês nas próximas semanas, se a estrutura não for bem pensada desde o início.

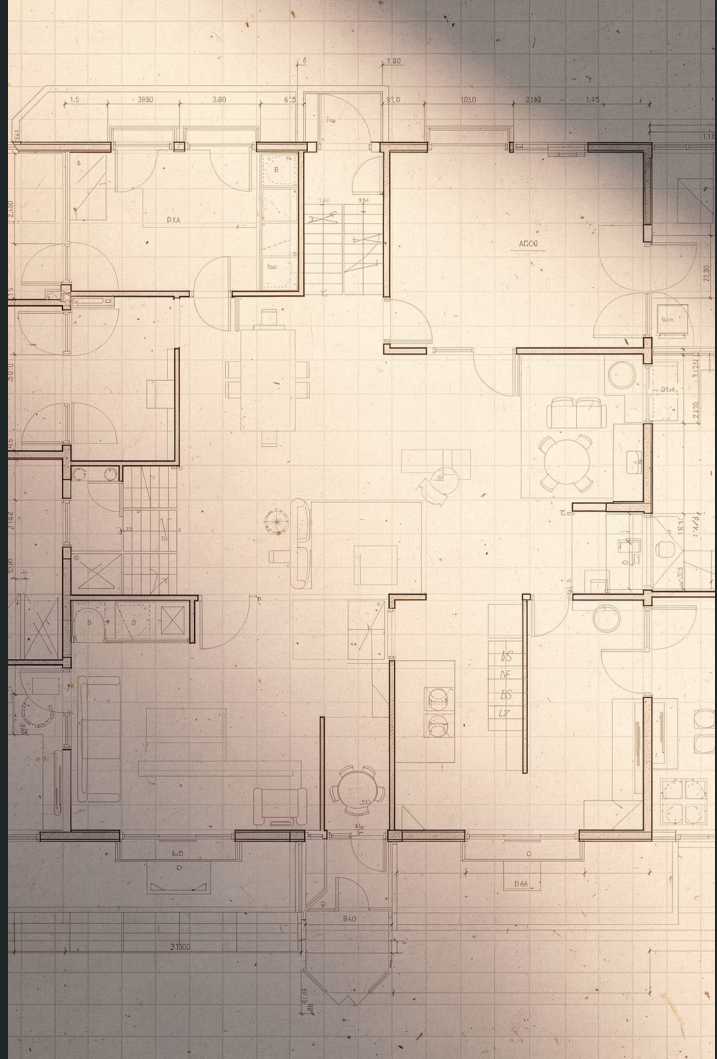




Bloco 2

Princípios de Design — Formalizando o que Você Já Sentiu

A discussão sobre os dois códigos não foi acidental. Ela revelou princípios reais de design de software. Agora vamos nomeá-los e entendê-los com precisão.



Princípio 1 — Separação de Responsabilidades

Cada parte do código deve ter uma única razão para existir. Se uma função faz validação, acesso ao banco e envio de e-mail, ela tem três razões para mudar — e qualquer uma delas pode quebrar as outras duas.



Routes

Definem quais URLs o sistema conhece. Só isso.



Controllers

Recebem a requisição e montam a resposta. Delegam o trabalho pesado.



Services

Contêm a lógica de negócio — a parte mais valiosa do sistema.



Repositories

Falam com o banco de dados. Isolar aqui significa que trocar o banco não afeta nada acima.



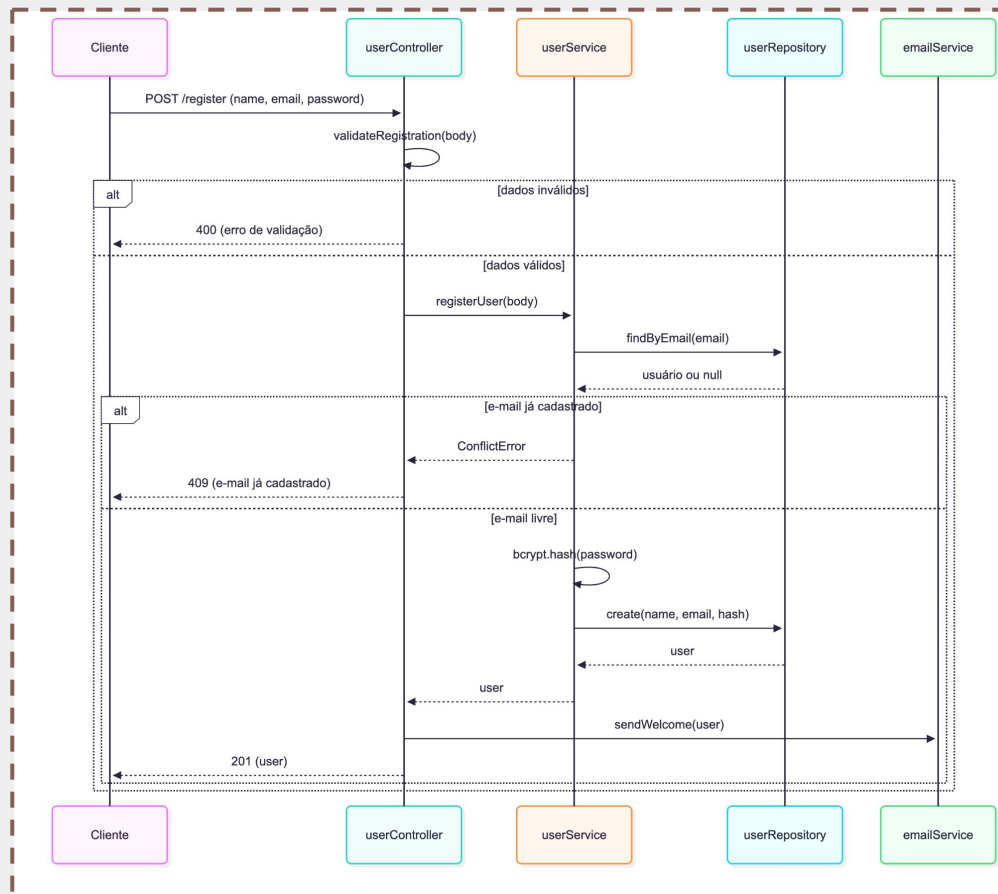
O caminho de uma requisição — POST /register (versão B)

Diagrama de sequência

Cada seta atravessa uma fronteira de responsabilidade. Nenhuma camada pula a camada vizinha.

Controller valida e responde
Service aplica a regra de negócio
Repository fala com o banco

É o caminho percorrido pelo código da Versão B, só que desenhado.



Princípio 2 — Coesão

"Este arquivo faz sentido como uma unidade?"

Coesão é a qualidade de um módulo onde **tudo que está dentro pertence ao mesmo assunto**. Se você abre um arquivo chamado `userService.js` e encontra uma função que envia e-mail, algo está errado. O nome prometia uma coisa, o conteúdo entregou outra.

✗ Coesão Baixa

`userService.js` com função de envio de e-mail

`utils.js` com 800 linhas de funções sem relação entre si

- Controller que valida dados, consulta banco e formata PDF

✓ Coesão Alta

`userService.js` só tem lógica de usuário

`emailService.js` só tem lógica de envio de e-mail

- Controller que recebe, delega e responde — nada mais

Teste rápido: se você consegue resumir o propósito de um arquivo em uma frase curta, ele provavelmente tem boa coesão.



Princípio 3 — Acoplamento

Acoplamento é o oposto da **independência**. Quanto mais uma parte do sistema precisa conhecer os detalhes internos de outra, maior o acoplamento — e mais frágil o sistema se torna.

Alto acoplamento



Ideal:
Controller→Service→R
epository limpo



Ruim: Controller acessa
o banco direto

Interfaces claras



Péssimo: Camadas
interdependentes
cruzadas



Mitigação: Isolar
responsabilidades e
interfaces

Dependência direta

O ideal é que cada camada conheça apenas a **interface** da camada imediatamente abaixo — não os detalhes internos. Se amanhã o banco mudar, só o Repository precisa ser atualizado. O Controller não sabe que o banco

Baixo acoplamento

existe.



Princípio 4 — Abstração e Nomes Significativos

"Nomes bons são a forma mais barata de documentação que existe."

O que esses nomes comunicam?

```
// Versão ruim
const d = new Date();
const x = d.getTime() - t;
if (x > 86400000) { ... }

// Versão boa
const now = new Date();
const elapsedMs = now.getTime() -
  sessionStartTime;
const oneDayMs = 86400000;
if (elapsedMs > oneDayMs) { ... }
```

Por que isso importa

Na versão ruim, `x` não diz nada. Você precisa rastrear mentalmente o que `t` representa, calcular o que 86400000 significa e deduzir o propósito da comparação.

Na versão boa, o código **se explica**. Um colega que nunca viu esse arquivo entende o que acontece em 5 segundos.

- ❏ Se você abre um PR e vê uma variável chamada `x` usada 15 linhas abaixo — isso é um problema de comunicação, não só de estilo.



Os Quatro Princípios — Resumo Visual



Separação de Responsabilidades

Uma razão para existir. Uma razão para mudar.



Coesão

Tudo dentro de um módulo fala sobre o mesmo assunto.



Acoplamento

Conheça apenas a interface — nunca os detalhes internos.



Nomes Significativos

O código que se explica não precisa de comentário.





Bloco 3

Conectando com o Projeto Real

Princípios sem aplicação são abstrações. Vamos traduzir o que aprendemos em decisões concretas que a equipe vai tomar no projeto



Estrutura de Pastas Sugerida — Backend

Esta não é uma estrutura arbitrária. Cada pasta representa uma responsabilidade clara, e o nome de cada diretório comunica o que mora lá dentro.

Estrutura

```
backend/  
└─ src/  
    ├── routes/  
    ├── controllers/  
    ├── services/  
    ├── repositories/  
    └─ utils/
```

O que mora em cada camada

routes/

O Express aprende que `/api/users` existe. Só isso.

controllers/

A requisição chega aqui, é delegada ao service e a resposta é montada.

services/

A lógica de negócio — a parte mais valiosa do código.

repositories/

Trocou de SQLite para PostgreSQL? Só esta camada muda.



Estrutura de Pastas — Frontend e Docs

Frontend

frontend/

└─ src/

└─ pages/

└─ components/

└─ hooks/

└─ services/

- **pages/** — Telas completas da aplicação
- **components/** — Blocos reutilizáveis de UI
- **hooks/** — Lógica de estado e efeitos reutilizável
- **services/** — Chamadas à API do backend

Documentação

docs/

└─ diagramas C4

└─ ADRs

└─ README

A pasta docs/ não é opcional. É onde a equipe registra **por que** tomou as decisões de arquitetura, não só o quê.

- 📄 Na quinta-feira, o Workshop de Arquitetura vai exigir que vocês documentem essas decisões. Hoje é o momento de pensar — quinta é o momento de decidir e implementar.



Compasso — nosso sistema de referência

O que é

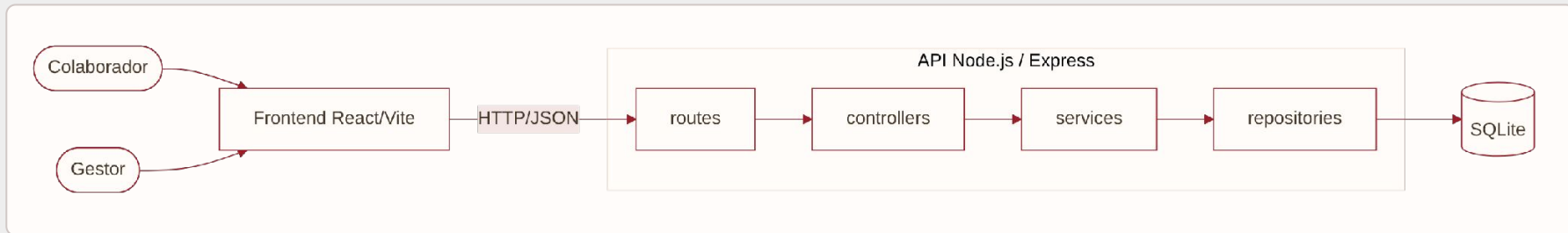
Plataforma de time tracking: horas trabalhadas por projeto

Atores

Colaborador registra horas. Gestor aprova e vê relatórios por projeto

Stack

React/Vite + Node.js/Express + SQLite



Rota central: POST /time-entry

Não confundir com POST /register, que pertence ao exercício dos dois códigos.



ADR — Registro de Decisão de Arquitetura

ADR-00X: [Título da decisão]

Status:

Proposto, aceito ou substituído?

Contexto:

Que situação nos obriga a decidir?

Decisão:

O que vamos fazer, e por quê?

Consequências:

O que fica mais fácil e mais difícil?

Um ADR registra o porquê.

O código só mostra o quê.

Formato Nygard

Título e quatro seções. Fica versionado em docs/ADR-00X.md, junto do código.

Agora vamos preencher um, juntos, para o Compasso.



ADR-001 — Usar SQLite como banco de dados

Status	Aceito
Contexto	sistema de time tracking, projeto de 4 semanas, equipe de 4 pessoas
Decisão	Usaremos SQLite. Motivo: sem infraestrutura de servidor; volume de registros é baixo e previsível
Consequências	Mais fácil: setup imediato, sem servidor de banco para administrar. Mais difícil: não escala para produção com múltiplos projetos; aceito para este contexto.

Arquivo: docs/ADR-001.md



Quais Serviços o Projeto de Vocês Já Tem?

Olhando para o backlog do projeto de vocês, conseguem identificar as camadas de serviço que já existem?

Discutam em equipe por 5 minutos.

UserService

Cadastro, autenticação, perfil. Presente em praticamente todo projeto.

TaskService / ItemService

A entidade principal do negócio — depende do domínio do projeto.

NotificationService

E-mails, alertas, push notifications — separar isso evita acoplamento com lógica de negócio.

AuthService

Geração e validação de tokens. Isolado, pode evoluir sem tocar no resto do sistema.

Anotem no material da equipe os serviços que identificaram. Esta lista vai ser a base do Workshop de Arquitetura.





Bloco 4 — A Questão

"Como a organização interna de um sistema influencia a capacidade de uma equipe de mantê-lo e evoluí-lo ao longo do tempo?"

2 minutos de reflexão individual. 3 minutos em dupla. Depois, compartilhamos com a turma.



Evidências que Você Já Tem

Quando a turma compartilhar, o professor vai provocar: *"Vocês têm evidência disso? Onde viram isso acontecer?"*

Se ninguém tiver vivência própria — conectem com o exercício dos dois códigos. Imaginem a Versão A depois de 8 semanas com 4 pessoas adicionando features sem parar.



O custo invisível do código desorganizado

Cada nova feature na Versão A adiciona mais linhas ao mesmo arquivo monolítico. O tempo para entender o código antes de mexer nele cresce semana a semana.



O dividend da organização

Na Versão B, o time pode crescer sem caos. Novos membros encontram o lugar certo para cada coisa. O sistema resiste ao tempo porque foi projetado para isso.



Fechamento — O que Vem a Seguir

"Hoje vocês viram **por que** organização importa. Em breve vocês vão aprender uma forma de **documentar** essa organização visualmente."

1

Hoje

Organização do código — princípios e estrutura de pastas para o projeto.

2

Workshop de Arquitetura

Documentar e implementar as decisões tomadas pela equipe.

3

Semana 7

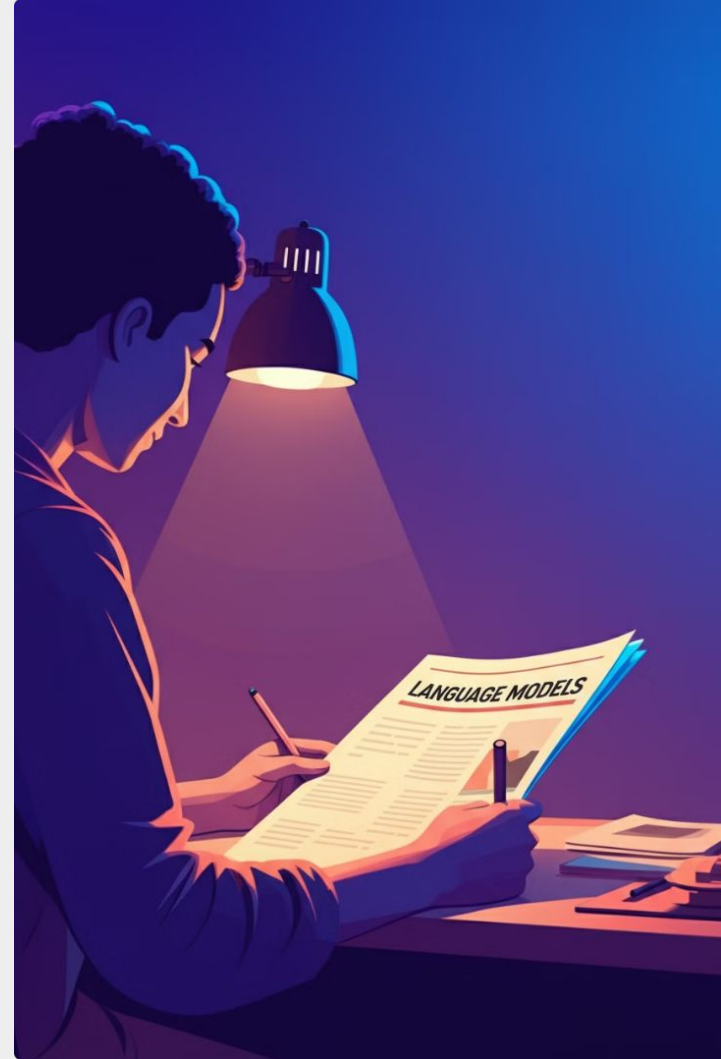
Estilos arquiteturais e C4 Model — transformar arquitetura abstrata em diagramas concretos que toda a equipe lê.

- ❑ **Leitura para a Semana 7:** Cap. 14, seções 14.3.1 a 14.3.3 (C4 Model) e 14.4 (C4 vs UML). Tragam um esboço mental dos atores e blocos técnicos do projeto de vocês.



Próximos passos

- Eng. Software em Dimensões
 - Cap. 14 (seções 14.1–14.8):
Arquitetura, C4 Model,
qualidade arquitetural
- Eng. Software Moderna
 - "Cap. 5 — Princípios de
Projeto Coesão, acoplamento,
SOLID
 - Cap. 7 — Arquitetura Camadas,
MVC, trade-offs"



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://www.instagram.com/vinicius3w)
viniciusgarcia.me

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Princípios de design de software